Handling Date and Time Variables

Extraction, arithmetic and cyclical encoding

Contents

Handling Date and Time Variables 2
34.1 Dates arrive as strings 2
34.2 The dt accessor 2
34.3 Which parts are features? 4
34.4 Date arithmetic 4
34.5 Boolean calendar flags 4
34.6 Cyclical encoding 5
34.7 Putting it in a pipeline 6
34.8 The lecture's walkthrough, summarised 6
34.9 Recap, practice and sources 6

CHAPTER 34

Handling Date and Time Variables

Extraction, arithmetic and cyclical encoding

> **WHAT THIS CHAPTER BUILDS**
>
> A timestamp is a compressed bundle: year, month, weekday, hour, and the intervals between events. This chapter unpacks the bundle with the pandas dt accessor, does date arithmetic correctly, and encodes cyclic features so that the end of December is close to the beginning of January.

34.1 Dates arrive as strings

In real data a date column is usually text: `"2019-12-12"`, `"08/08/2018"`, `"12 Dec 2019"`. Before any feature can be built, pandas must parse it into a `datetime64` dtype.

```
import pandas as pd

df = pd.DataFrame({
    "signup": ["2019-12-12", "2018-08-08", "2020-01-31"],
    "visit":  ["12/01/2020", "08/08/2018", "31/12/2019"],
})
df["signup"] = pd.to_datetime(df["signup"])
df["visit"] = pd.to_datetime(df["visit"], dayfirst=True)
```

`pd.to_datetime` does three jobs: it detects the format, it converts to a datetime dtype, and it raises when a value cannot be parsed. Two options control that behaviour. `errors="coerce"` turns unparseable values into `NaT` instead of raising; the result must then be treated as missing. `format` pins the expected pattern so that ambiguous strings cannot be misread: `"08/08/2018"` could be August 8 or 8 August, and only the format or `dayfirst` can decide.

```
bad = pd.to_datetime(["2019-12-12", "not-a-date"], errors="coerce")
print(bad)    # NaT for the second element
```

`NaT` ("Not a Time") is the datetime counterpart of `NaN`. It propagates through date arithmetic and must be imputed or dropped like any other missing value.

> **pandas 3.0 parsing is stricter**
>
> This environment runs pandas 3.0.5. Format detection is stricter than in older releases: for a column with genuinely mixed formats, pass `format="mixed"` so pandas parses each element with its own pattern instead of inferring one format for the whole column. The lecture's code was written on an older pandas; check the parser options before assuming the lecture's snippet runs unchanged.

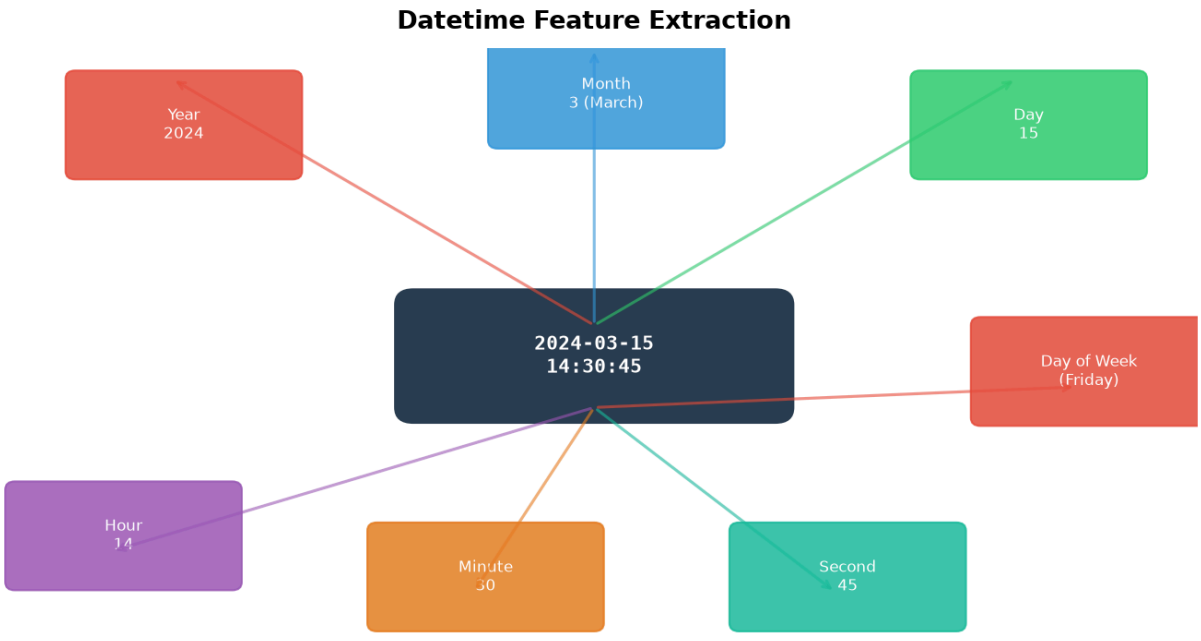
34.2 The dt accessor

Once the column has datetime dtype, the `dt` accessor exposes every component. The lecture works through these accessors line by line; the table below is the working summary.

Expression	Returns	Example for 2019-12-12
`dt.year`	int	2019
`dt.month`	int	12
`dt.day`	int	12
`dt.dayofweek`	int, Monday = 0	3
`dt.day_name()`	string	Thursday
`dt.quarter`	int	4
`dt.weekday`	int, Monday = 0	3
`dt.isocalendar().week`	int	50
`dt.days_in_month`	int	31
`dt.is_month_start`	bool	False
`dt.is_month_end`	bool	True

The companion expressions are `dt.date`, which strips the time and returns Python `date` objects; `dt.time`, which keeps only the time; and `dt.hour`, `dt.minute`, `dt.second`, which decompose the time part. For a column with no time component, every time accessor returns 0 rather than an error.

> **dt.week is gone in pandas 3**
>
> Older code uses `s.dt.week` for the ISO week number. That attribute no longer exists in pandas 3.0; it was always ISO week numbering in disguise. Use `s.dt.isocalendar().week` instead, which also returns the ISO year and day so the week number cannot be mislabelled across a year boundary.



One timestamp decomposes into calendar components. Each component can become its own feature, and components that repeat on a cycle need a circular encoding.

34.3 Which parts are features?

Not every component deserves a column. The question is whether the component carries signal for the task.

Year is often a proxy for drift: prices, habits, and policies change over time. **Month** and **quarter** capture seasonality. **Weekday** separates business days from weekends. **Hour** separates morning from night. Each of these can be treated as a categorical feature (with one-hot encoding) or as a cyclic numeric feature (with sine and cosine, explained in 34.6).

Day and **week of the year** are usually weak on their own but useful in interactions, such as "days since launch". **Day name** is a re-expression of `dayofweek` and should not be added alongside it; that would duplicate the same information.

A date can also be a label rather than a feature. In a time-series split, the timestamp determines the order of the split, not a predictor column. Confusing the two is leakage: a model must never see the future to predict the past.

34.4 Date arithmetic

The value of a date column often lies in the interval between two dates: how long since signup, how old the account is, how many days until expiry. pandas represents an interval as `Timedelta`.

```
today = pd.Timestamp("2020-01-31")

df["days_since_signup"] = (today - df["signup"]).dt.days
df["age_days"] = (pd.Timestamp.now() - df["visit"]).dt.days
```

Subtracting two datetime Series produces a `Timedelta` Series. `.dt.days` gives whole days; `.dt.total_seconds()` gives seconds as a float, which is the right unit when sub-day precision matters. The lecture uses the same pattern to obtain the days, hours, minutes, and seconds between two timestamps.

Converting to months needs care. `np.timedelta64(1, "M")` is not a fixed number of days: a calendar month has 28, 29, 30, or 31 days, so dividing a `Timedelta` by a month unit silently assumes an average length. For a stable feature, divide days by `365.25`, or use `dateutil.relativedelta` when the exact calendar month count matters.

> **now() is not reproducible**
>
> `pd.Timestamp.now()` makes a feature that changes every time the script runs. In a pipeline, compute ages against a fixed reference date chosen at training time, or store the reference date with the model. Otherwise the same row produces different features today and tomorrow.

34.5 Boolean calendar flags

Small boolean features are cheap and often strong. The lecture derives weekend and semester flags; the rest of this family follows the same pattern.

```
df["is_weekend"] = df["signup"].dt.dayofweek >= 5
df["is_month_end"] = df["signup"].dt.is_month_end
df["is_quarter_end"] = df["signup"].dt.is_quarter_end
df["is_year_end"] = df["signup"].dt.is_year_end
```

```
df["is_leap_year"] = df["signup"].dt.is_leap_year
df["is_south_semester"] = df["signup"].dt.month.isin([1, 2, 7, 8])
```

`dayofweek >= 5` marks Saturday and Sunday because the accessor is Monday = 0, Sunday = 6. The semester expression is domain-specific: define it from the business calendar you actually care about, not from a generic rule. If the calendar varies by country or year, store it as an explicit lookup table rather than encoding it in a chain of `isin` calls.

34.6 Cyclical encoding

Month 12 and month 1 are adjacent in the calendar but 11 apart as integers. A model that treats month as a number will learn two unrelated effects, and the jump at the year boundary is an artefact. The fix is to map the cycle onto a circle: each value becomes a point $(\cos\theta, \sin\theta)$, where

$$\theta = 2\pi \frac{m-1}{K}$$

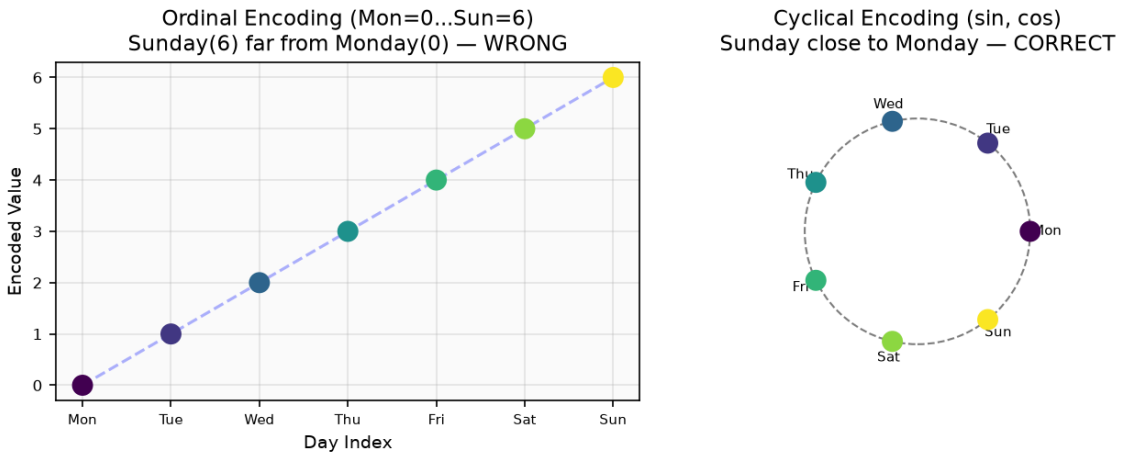
for K positions in the cycle. December $(m = 12)$ and January $(m = 1)$ land next to each other on the circle, so their Euclidean distance is small.

```
import numpy as np

month = df["signup"].dt.month
df["month_sin"] = np.sin(2 * np.pi * (month - 1) / 12)
df["month_cos"] = np.cos(2 * np.pi * (month - 1) / 12)

hour = df["visit"].dt.hour
df["hour_sin"] = np.sin(2 * np.pi * hour / 24)
df["hour_cos"] = np.cos(2 * np.pi * hour / 24)
```

The same trick works for weekday (7 positions), day of month (31), and any other repeating cycle. The result is two continuous columns. One-hot encoding remains an alternative when the number of levels is small and the effect of each level is genuinely unrelated; sine and cosine impose a smooth relationship that may be wrong if, say, only Mondays behave differently.



An integer cycle has an artificial jump between the last and first value. A sine and cosine pair places the same values on a circle, where the ends are neighbours.

34.7 Putting it in a pipeline

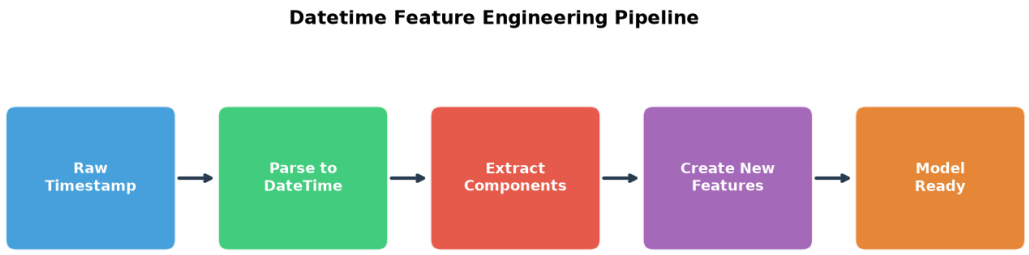
Date extraction is deterministic: `dt.year` needs no learned state. That makes it safe to run outside a fitted transformer, but the **decision** of which components to extract is still a modelling choice made on training data, and any imputation or encoding of the extracted features must be fitted inside the pipeline.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

df["weekday"] = df["signup"].dt.day_name()      # stateless extraction

preprocess = ColumnTransformer([
    ("weekday", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), ["weekday"]),
])
```

For production, three rules keep the features trustworthy. Parse with an explicit format so a locale change cannot flip day and month. Handle time zones before extracting hour and weekday; `"2019-12-12 23:00"` in one zone is a different weekday-hour pair in another. And forbid any feature that could not have been known at prediction time: “days since last login” is fine, “days until the campaign ended” is not.



Extraction is stateless, but the components it produces enter a fitted pipeline for imputation and encoding just like any other feature.

34.8 The lecture’s walkthrough, summarised

The lecture’s code follows the path described above. It converts strings with `pd.to_datetime`, extracts `.dt.year`, `.dt.month`, `.dt.day`, `.dt.dayofweek` and `.dt.day_name()`, checks `.dt.quarter` and the week number, builds weekend and semester flags, and then computes intervals with `.dt.total_seconds()` and the unit-specific `.dt.days`. For finer parts, `.dt.components` exposes `hours`, `minutes`, `seconds` and the sub-second fields; note that it does **not** expose calendar months, because a `Timedelta` is an exact duration, not a calendar span. The goal throughout is to make the calendar legible to a model: each useful component becomes an explicit column instead of a string the model cannot parse.

34.9 Recap, practice and sources

The chain is: parse strings into datetime dtype with an explicit format and `errors` policy; decompose with `dt`; decide which components are features; compute intervals as timedeltas and choose a stable unit;

add boolean calendar flags; encode repeating cycles with sine and cosine; keep the timestamp itself as the ordering, not the predictor, in time-based splits.

> **Lock-in line**
>
> A date is not one feature. It is a coordinate system, and each component answers a different question about when something happened.

Practice

1. Parse the mixed-format column with and without `format="mixed"`. Which rows differ?
2. Extract `dayofweek` for a two-week sample. Verify Monday = 0 and locate both weekends.
3. Replace `dt.week` with `dt.isocalendar().week` and confirm the ISO year column.
4. Compute days since signup against a fixed reference date. Compare with `Timestamp.now()`.
5. Encode month as one-hot and as sine/cosine. Show the distance between December and January under both representations.

Sources and scope

Video 34, “Handling Date and Time Variables”, CampusX 100 Days of Machine Learning. The lecture is a pandas code walkthrough of datetime conversion, dt extraction, calendar flags, and date arithmetic. The cyclical encoding, the time-zone warning, and the pandas 3.0 compatibility notes are supplementary explanations added here; they are consistent with the lecture’s mechanics.